JavaStructureAnalyzer.java 解説

JavaParser を使って、フォルダ内の .java ファイルの「構造（形式）」を一覧表示するプログラムを、1 行ずつ説明します。

このプログラムは何をする？

指定したフォルダ（サブフォルダも含む）にある `.java` ファイルを全部読み込み、それぞれについて次の内容を表示します。

- package 名と import の一覧
- クラス・インターフェース・enum・record・アノテーション型の宣言（修飾子、extends / implements など）
- フィールド・コンストラクタ・メソッドの形（型、名前、引数、throws）
- 入れ子になったクラスも、インデントを付けて表示

実行方法

```
./gradlew :app:parseJava --args="app/src/main/java"
```

出力例

```
==== org/example/EigenWasm.java ====
package: org.example
imports:
  org.mklab.nfc.eig.DoubleRealEigenSolver
  ...
final class EigenWasm
  [constructor] private EigenWasm()
  [method] static double[] eigenValues(double[] values, int size)
  [method] static double[] eigen(double[] values, int size)
```

処理の流れ

main
引数からフォルダを決めて、存在するか確認

→

analyzeDirectory
フォルダをたどって .java を集め、1 つずつ処理

→

analyzeFile
JavaParser で構文木を作り、package / import を表示

→

printType
クラスの見出しとメンバーを表示（入れ子なら自分を再帰呼び出し）

見出しの文字列を作るときは、`kindOf` と `append〇〇` / `joinParameters` の補助メソッドを使います。

先に知っておくとよい用語

構文木（AST）
ソースコードを「ファイル → クラス → メソッド → 引数…」という木の形に整理したデータ。JavaParser は

instanceof パターン
`if (x instanceof MethodDeclaration m)` と書くと、型を調べると同時に、その型の変数 `m` としてすぐ

	.java ファイルを読んでこれを作ります。根っこが `CompilationUnit` です。
ノード	木の 1 つ 1 つの要素。`MethodDeclaration`（メソッド）や `FieldDeclaration`（フィールド）など、種類ごとにクラスが分かれています。
Stream	`.filter()`（絞り込み）、`.map()`（変換）、`.collect()`（まとめる）を順につなげてデータを処理する書き方。
Optional	「値があるかもしれないし、無いかもしれない」箱。`.map()` / `.orElse()` / `.ifPresent()` で安全に取り出します。

	使えます（Java 16 以降）。
StringBuilder	文字列を少しずつ書き足していく道具。`.append()` でつなげ、最後に表示します。
再帰	メソッドが自分自身を呼び出すこと。`printType` は入れ子のクラスを見つけると、深さ +1 で自分を呼び出します。
三項演算子	`条件 ? A : B`。条件が真なら A、偽なら B になります。

構文木のイメージ

```
package org.example;
import java.util.List;

public class Sample {
    private int count;
    public void add(int n) { ... }
}
```

```
CompilationUnit                    ← ファイル全体
├─ PackageDeclaration  org.example
├─ ImportDeclaration   java.util.List
└─ ClassOrInterfaceDeclaration  Sample
   ├─ FieldDeclaration   private int count
   └─ MethodDeclaration  public void add(int n)
```

ソースコードの 1 行ずつの説明

① パッケージ宣言と import（1〜34 行）

このファイルで使う他のクラスを読み込みます。前半が Java 標準ライブラリ、後半が JavaParser ライブラリのクラスです。JavaParser のクラスの多くは「構文木（AST）のノードの種類」を表しています。

行	コード	説明
1	`package JavaParser;`	このファイルが属するパッケージ（フォルダ名）を宣言。src/main/java/JavaParser/ に置いているので「JavaParser」。
2		
3	`import java.io.IOException;`	ファイル読み込み時のエラーを表す例外クラス IOException を使えるようにする。
4	`import java.nio.file.Files;`	Files：ファイルやフォルダを操作する便利メソッド集（存在確認・フォルダ探索など）。
5	`import java.nio.file.Path;`	Path：ファイルやフォルダの「場所（パス）」を表す型。
6	`import java.nio.file.Paths;`	Paths：文字列から Path を作るためのクラス。
7	`import java.util.List;`	List：順番付きのデータの並び（リスト）を表す型。
8	`import java.util.stream.Collectors;`	Collectors：Stream の結果をリストや文字列にまとめる道具。
9	`import java.util.stream.Stream;`	Stream：データの流れに対して filter / map などを順に適用する仕組み。
10		
11	`import com.github.javaparser.ParseResult;`	ParseResult：JavaParser の解析結果（成功/失敗と、得られた構文木）を入れる箱。
12	`import com.github.javaparser.ParserConfiguration;`	ParserConfiguration：JavaParser の設定（どの Java バージョンの文法で読むか等）。
13	`import com.github.javaparser.ParserConfiguration.LanguageLevel;`	LanguageLevel：対応する Java のバージョンを表す列挙型（JAVA_21 など）。
14	`import com.github.javaparser.Problem;`	Problem：解析に失敗したときのエラー内容 1 件を表す。
15	`import com.github.javaparser.ast.CompilationUnit;`	CompilationUnit：1 つの .java ファイル全体を表す構文木（AST）の根っこ。
16	`import com.github.javaparser.ast.ImportDeclaration;`	ImportDeclaration：import 文 1 行を表すノード。
17	`import com.github.javaparser.ast.Modifier;`	Modifier：public / static / final などの修飾子を表すノード。

行	コード	説明
18	`import com.github.javaparser.ast.NodeList;`	NodeList：JavaParser 独自のリスト。構文木の子ノードの並びに使われる。
19	`import com.github.javaparser.ast.body.AnnotationDeclaration;`	AnnotationDeclaration：@interface（アノテーション型）の宣言を表す。
20	`import com.github.javaparser.ast.body.AnnotationMemberDeclaration;`	AnnotationMemberDeclaration：アノテーション型の中の要素（例: String value();）を表す。
21	`import com.github.javaparser.ast.body.BodyDeclaration;`	BodyDeclaration：クラスの中身（フィールド・メソッドなど）の共通の親クラス。
22	`import com.github.javaparser.ast.body.ClassOrInterfaceDeclaration;`	ClassOrInterfaceDeclaration：class または interface の宣言を表す。
23	`import com.github.javaparser.ast.body.ConstructorDeclaration;`	ConstructorDeclaration：コンストラクタの宣言を表す。
24	`import com.github.javaparser.ast.body.EnumConstantDeclaration;`	EnumConstantDeclaration：enum の定数 1 つ（例: RED）を表す。
25	`import com.github.javaparser.ast.body.EnumDeclaration;`	EnumDeclaration：enum の宣言を表す。
26	`import com.github.javaparser.ast.body.FieldDeclaration;`	FieldDeclaration：フィールド（メンバ変数）の宣言を表す。
27	`import com.github.javaparser.ast.body.MethodDeclaration;`	MethodDeclaration：メソッドの宣言を表す。
28	`import com.github.javaparser.ast.body.Parameter;`	Parameter：メソッドやコンストラクタの引数 1 つを表す。
29	`import com.github.javaparser.ast.body.RecordDeclaration;`	RecordDeclaration：record（Java 16 以降のデータ専用クラス）の宣言を表す。
30	`import com.github.javaparser.ast.body.TypeDeclaration;`	TypeDeclaration：class / interface / enum / record / @interface すべての共通の親。
31	`import com.github.javaparser.ast.body.VariableDeclarator;`	VariableDeclarator：「int a, b;」の a や b のような、変数 1 個分を表す。
32	`import com.github.javaparser.ast.expr.AnnotationExpr;`	AnnotationExpr：@Override などのアノテーションの使用箇所を表す。
33	`import com.github.javaparser.ast.type.ClassOrInterfaceType;`	ClassOrInterfaceType：extends / implements の後ろに書かれる型（例: List<String>）。
34	`import com.github.javaparser.ast.type.TypeParameter;`	TypeParameter：型パラメータ（例: <T extends Number> の T）を表す。
35		

② クラスの説明とフィールド（36〜48 行）

クラス全体の説明（Javadoc）と、クラスが持つデータ（フィールド）です。

行	コード	説明
36	`/**`	Javadoc コメントの始まり。ここから 43 行目までがクラスの説明文。
37	`* JavaParser を使って、指定フォルダ以下の .java ファイルの構造（形式）を出力するツールです。`	このクラスの目的：フォルダ内の .java ファイルの構造を出力するツールであること。
38	`*`	Javadoc の空行（段落の区切り）。
39	`* <p>各ファイルについて、パッケージ・import・型宣言（class / interface / enum / record / annotation）、`	<p> は Javadoc 内の段落。何を表示するかを説明している。
40	`* 継承関係、フィールド、コンストラクタ、メソッドのシグネチャを表示します。入れ子の型も再帰的に表示します。</p>`	（39 行目の続き）入れ子のクラスも再帰的に表示することを説明。
41	`*`	Javadoc の空行。
42	`* <p>使い方: {@code ./gradlew :app:parseJava --args="<フォルダ>"}（フォルダ省略時はカレントディレクトリ）</p>`	使い方の説明。Gradle の parseJava タスクにフォルダを渡して実行する。
43	`*/`	Javadoc コメントの終わり。
44	`public final class JavaStructureAnalyzer {`	クラス宣言。public = どこからでも使える、final = 継承（extends）できないクラス。
45		
46	`    private static final String INDENT = "  ";`	インデント用の文字列（半角スペース 2 つ）。static final なので定数として全体で共有。
47		
48	`    private final com.github.javaparser.JavaParser parser;`	JavaParser 本体を保持するフィールド。自分のパッケージ名も JavaParser なので、混同しないように完全修飾名で書いている。
49		

③ コンストラクタ（50〜54 行）

JavaParser を「Java 21 の文法で読む」設定で準備します。

行	コード	説明
50	`    public JavaStructureAnalyzer() {`	コンストラクタ。new JavaStructureAnalyzer() したときに 1 回だけ実行される。
51	`        ParserConfiguration config = new ParserConfiguration()`	パーサーの設定オブジェクトを作る。
52	`                .setLanguageLevel(LanguageLevel.JAVA_21);`	「Java 21 の文法で読む」と指定（record や switch パターンなどの新しい文法に対応させるため）。
53	`        this.parser = new com.github.javaparser.JavaParser(config);`	設定を渡して JavaParser 本体を作り、フィールド parser に保存する。
54	`    }`	コンストラクタの終わり。
55		

④ main メソッド：プログラムの入口（56〜63 行）

コマンドライン引数からフォルダを決め、存在を確認してから解析を始めます。

行	コード	説明
56	`    public static void main(String[] args) throws IOException {`	プログラムの入口（main メソッド）。args にはコマンドラインで渡した引数が入る。
57	`        Path root = Paths.get(args.length > 0 ? args[0] : ".").toAbsolutePath().normalize();`	引数があれば 1 つ目を、無ければ "."（現在のフォルダ）を使う。toAbsolutePath で絶対パスに、normalize で「..」などを整理する。
58	`        if (!Files.isDirectory(root)) {`	指定されたパスがフォルダとして存在しない場合…
59	`            System.err.println("フォルダが見つかりません: " + root);`	エラーメッセージを標準エラー出力に表示する。
60	`            System.exit(1);`	終了コード 1（異常終了）でプログラムを終える。
61	`        }`	if の終わり。
62	`        new JavaStructureAnalyzer().analyzeDirectory(root);`	解析器を作り、フォルダの解析を開始する。
63	`    }`	main メソッドの終わり。
64		

⑤ analyzeDirectory：フォルダ内の .java を集める（65〜83 行）

フォルダをサブフォルダまでたどって .java ファイルを集め、1 つずつ analyzeFile に渡します。

行	コード	説明
65	`/** フォルダ以下の .java ファイルをすべて解析して出力します。 */`	メソッドの説明コメント（Javadoc）。
66	`public void analyzeDirectory(Path root) throws IOException {`	フォルダを受け取り、その中の .java ファイルをすべて解析するメソッド。
67	`    List<Path> files;`	見つかったファイルの一覧を入れる変数を用意。
68	`    try (Stream<Path> walk = Files.walk(root)) {`	Files.walk でフォルダ以下（サブフォルダも含む）を全部たどる。try (...) で書くと、使い終わったら自動で後片付け（close）される。
69	`        files = walk.filter(Files::isRegularFile)`	そのうち「普通のファイル」だけを残す（フォルダは除外）。
70	`                .filter(p -> p.toString().endsWith(".java"))`	さらに、名前が .java で終わるものだけを残す。
71	`                .sorted()`	パスの順に並べ替える（出力順を毎回同じにするため）。
72	`                .collect(Collectors.toList());`	結果をリストにまとめて files に入れる。
73	`    }`	try ブロックの終わり。ここで walk が自動で閉じられる。
74		
75	`    System.out.println("対象フォルダ: " + root);`	対象フォルダを表示。
76	`    System.out.println("Java ファイル数: " + files.size());`	見つかった Java ファイルの数を表示。
77		
78	`    for (Path file : files) {`	ファイルを 1 つずつ取り出して繰り返す。
79	`        System.out.println();`	空行を出力して、ファイルごとの区切りを見やすくする。
80	`        System.out.println("==== " + root.relativize(file) + " ====");`	「==== ファイル名 ====」の見出しを表示。relativize で、対象フォルダからの相対パスにしている。
81	`        analyzeFile(file);`	そのファイルを解析して内容を表示する。
82	`    }`	for の終わり。
83	`}`	analyzeDirectory メソッドの終わり。
84		

⑥ analyzeFile：1 ファイルを解析する（85〜112 行）

ファイルを構文木に変換し、package と import を表示したあと、中のクラスなどを printType に渡します。

行	コード	説明
85	`    /** 1 ファイルを解析して出力します。 */`	メソッドの説明コメント。
86	`    public void analyzeFile(Path file) throws IOException {`	1 つの .java ファイルを解析して表示するメソッド。
87	`        ParseResult<CompilationUnit> result = parser.parse(file);`	JavaParser でファイルを読み込み、構文木（AST）を作る。結果は ParseResult に入る。
88	`        if (!result.isSuccessful() || result.getResult().isEmpty()) {`	解析が失敗した、または結果が空の場合…
89	`            System.out.println("[解析エラー]");`	「[解析エラー]」と表示。
90	`            for (Problem problem : result.getProblems()) {`	見つかったエラー（Problem）を 1 つずつ取り出して…
91	`                System.out.println(INDENT + problem.getVerboseMessage());`	エラーの詳しい内容を表示する。
92	`            }`	for の終わり。
93	`            return;`	このファイルの処理はここで終了（return でメソッドを抜ける）。
94	`        }`	if の終わり。
95	`        CompilationUnit cu = result.getResult().get();`	解析結果から構文木の根っこ（CompilationUnit）を取り出す。
96		
97	`        System.out.println("package: " + cu.getPackageDeclaration()`	package 宣言を取り出す。結果は Optional（あるかもしれないし、無いかもしれない箱）。
98	`                .map(pd -> pd.getNameAsString())`	package 宣言があれば、その名前（例: org.example）に変換する。
99	`                .orElse("(default)"));`	package 宣言が無ければ「(default)」と表示する。
100		
101	`        if (!cu.getImports().isEmpty()) {`	import 文が 1 つ以上ある場合だけ…
102	`            System.out.println("imports:");`	「imports:」という見出しを表示。
103	`            for (ImportDeclaration imp : cu.getImports()) {`	import 文を 1 つずつ取り出して繰り返す。
104	`                System.out.println(INDENT + (imp.isStatic() ? "static " : "")`	static import なら先頭に「static 」を付ける（三項演算子 条件 ? A : B）。
105	`                        + imp.getNameAsString() + (imp.isAsterisk() ? ".*" : ""));`	import する名前を付け、「import xxx.*;」の形なら末尾に「.*」を付けて表示。

行	コード	説明
106	`            }`	for の終わり。
107	`        }`	if の終わり。
108		
109	`        for (TypeDeclaration<?> type : cu.getTypes()) {`	ファイルに書かれた型（クラスなど）を 1 つずつ取り出して…
110	`            printType(type, 0);`	その型の中身を表示する。0 は入れ子の深さ（一番外側）。
111	`        }`	for の終わり。
112	`    }`	analyzeFile メソッドの終わり。
113		

⑦ printType：クラスなどの中身を表示する（114〜182 行）

このプログラムの中心です。型の見出し行を組み立てたあと、メンバーを種類（フィールド・コンストラクタ・メソッドなど）ごとに表示します。入れ子のクラスは自分自身を呼び出して（再帰）表示します。

行	コード	説明
114	`    private void printType(TypeDeclaration<?> type, int depth) {`	型 1 つ分（クラスなど）を表示するメソッド。depth は入れ子の深さで、インデント量に使う。
115	`        String pad = INDENT.repeat(depth);`	深さに応じたインデント文字列を作る（深さ 2 ならスペース 4 つ）。
116	`        StringBuilder header = new StringBuilder(pad);`	見出し行を組み立てるための StringBuilder（文字列をつなげていく道具）を用意。
117	`        appendAnnotations(header, type.getAnnotations());`	型に付いたアノテーション（例: @Deprecated）を追加。
118	`        appendModifiers(header, type.getModifiers());`	修飾子（public final など）を追加。
119	`        header.append(kindOf(type)).append(' ').append(type.getNameAsString());`	型の種類（class など）と、型の名前を追加。
120		
121	`        if (type instanceof ClassOrInterfaceDeclaration c) {`	型が class か interface なら…（instanceof の後ろの c は、その型として使える変数）
122	`            appendTypeParameters(header, c.getTypeParameters());`	型パラメータ（例: <T>）を追加。
123	`            appendTypeList(header, " extends ", c.getExtendedTypes());`	extends（継承元）を追加。
124	`            appendTypeList(header, " implements ", c.getImplementedTypes());`	implements（実装しているインターフェース）を追加。

行	コード	説明
125	`            appendTypeList(header, " permits ", c.getPermittedTypes());`	permits（sealed クラスで継承を許可しているクラス）を追加。
126	`        } else if (type instanceof RecordDeclaration r) {`	型が record なら…
127	`            appendTypeParameters(header, r.getTypeParameters());`	型パラメータを追加。
128	`            header.append('(').append(joinParameters(r.getParameters())).append(')');`	record の要素（例: (int x, int y)）を丸括弧で囲んで追加。
129	`            appendTypeList(header, " implements ", r.getImplementedTypes());`	implements を追加。
130	`        } else if (type instanceof EnumDeclaration e) {`	型が enum なら…
131	`            appendTypeList(header, " implements ", e.getImplementedTypes());`	implements を追加。
132	`        }`	if の終わり。
133	`        System.out.println(header);`	組み立てた見出し行を表示する。
134		
135	`        String inner = INDENT.repeat(depth + 1);`	メンバー用のインデント（型より 1 段深くする）。
136		
137	`        if (type instanceof EnumDeclaration e && !e.getEntries().isEmpty()) {`	enum で、定数が 1 つ以上あるなら…
138	`            System.out.println(inner + "[enum constants] " + e.getEntries().stream()`	「[enum constants]」に続けて定数の一覧を表示する。
139	`                    .map(EnumConstantDeclaration::getNameAsString)`	各定数をその名前に変換し…
140	`                    .collect(Collectors.joining(", ")));`	「, 」区切りで 1 つの文字列にまとめる。
141	`        }`	if の終わり。
142		
143	`        for (BodyDeclaration<?> member : type.getMembers()) {`	型の中のメンバー（フィールド・メソッドなど）を 1 つずつ取り出して、種類ごとに処理を分ける。
144	`            if (member instanceof FieldDeclaration f) {`	メンバーがフィールドなら…
145	`                for (VariableDeclarator v : f.getVariables()) {`	「int a, b;」のように 1 行で複数宣言されることがあるので、変数ごとに繰り返す。
146	`                    StringBuilder sb = new StringBuilder(inner).append("[field] ");`	「[field] 」から始まる行を作る。

行	コード	説明
147	`                appendAnnotations(sb, f.getAnnotations());`	アノテーションを追加。
148	`                appendModifiers(sb, f.getModifiers());`	修飾子を追加。
149	`                sb.append(v.getTypeAsString()).append(' ').append(v.getNameAsString());`	型と変数名（例: String name）を追加。
150	`                System.out.println(sb);`	フィールドの行を表示。
151	`            }`	内側の for の終わり。
152	`        } else if (member instanceof ConstructorDeclaration c) {`	メンバーがコンストラクタなら…
153	`            StringBuilder sb = new StringBuilder(inner).append("[constructor] ");`	「[constructor] 」から始まる行を作る。
154	`            appendAnnotations(sb, c.getAnnotations());`	アノテーションを追加。
155	`            appendModifiers(sb, c.getModifiers());`	修飾子を追加。
156	`            appendTypeParameters(sb, c.getTypeParameters());`	型パラメータを追加（ジェネリックなコンストラクタの場合）。
157	`            sb.append(c.getNameAsString())`	コンストラクタ名を追加。
158	`                    .append('(').append(joinParameters(c.getParameters())).append(')');`	引数の一覧を丸括弧で囲んで追加。
159	`            appendThrows(sb, c.getThrownExceptions());`	throws（投げる可能性のある例外）を追加。
160	`            System.out.println(sb);`	コンストラクタの行を表示。
161	`        } else if (member instanceof MethodDeclaration m) {`	メンバーがメソッドなら…
162	`            StringBuilder sb = new StringBuilder(inner).append("[method] ");`	「[method] 」から始まる行を作る。
163	`            appendAnnotations(sb, m.getAnnotations());`	アノテーション（例: @Override）を追加。
164	`            appendModifiers(sb, m.getModifiers());`	修飾子を追加。
165	`            if (!m.getTypeParameters().isEmpty()) {`	ジェネリックメソッド（例: <T> T get()）の場合だけ…
166	`                appendTypeParameters(sb, m.getTypeParameters());`	型パラメータを追加し…
167	`                sb.append(' ');`	後ろにスペースを 1 つ入れる。
168	`            }`	if の終わり。

行	コード	説明
169	`                sb.append(m.getTypeAsString()).append(' ').append(m.getNameAsString())`	戻り値の型とメソッド名を追加。
170	`                        .append('(').append(joinParameters(m.getParameters())).append(')');`	引数の一覧を丸括弧で囲んで追加。
171	`                appendThrows(sb, m.getThrownExceptions());`	throws を追加。
172	`                System.out.println(sb);`	メソッドの行を表示。
173	`            } else if (member instanceof AnnotationMemberDeclaration a) {`	メンバーがアノテーション型の要素なら…
174	`                StringBuilder sb = new StringBuilder(inner).append("[element] ");`	「[element] 」から始まる行を作る。
175	`                sb.append(a.getTypeAsString()).append(' ').append(a.getNameAsString()).append("()");`	要素の型・名前と「()」を追加。
176	`                a.getDefaultValue().ifPresent(d -> sb.append(" default ").append(d));`	default 値があれば「default 値」を追加（ifPresent = 値があるときだけ実行）。
177	`                System.out.println(sb);`	要素の行を表示。
178	`            } else if (member instanceof TypeDeclaration<?> nested) {`	メンバーが入れ子のクラスなどの型なら…
179	`                printType(nested, depth + 1);`	自分自身（printType）を深さ +1 で呼び出す（再帰）。これで何重の入れ子でも表示できる。
180	`            }`	if〜else if の終わり。
181	`        }`	for の終わり。
182	`    }`	printType メソッドの終わり。
183		

⑧ kindOf：型の種類を文字列にする（184〜195 行）

構文木ノードのクラスを調べて、"class" や "enum" などの文字列を返します。

行	コード	説明
184	`    private static String kindOf(TypeDeclaration<?> type) {`	型の種類を文字列で返すメソッド。
185	`        if (type instanceof ClassOrInterfaceDeclaration c) {`	class / interface の宣言なら…

行	コード	説明
186	`            return c.isInterface() ? "interface" : "class";`	interface なら "interface"、そうでなければ "class" を返す。
187	`        } else if (type instanceof EnumDeclaration) {`	enum なら…
188	`            return "enum";`	"enum" を返す。
189	`        } else if (type instanceof RecordDeclaration) {`	record なら…
190	`            return "record";`	"record" を返す。
191	`        } else if (type instanceof AnnotationDeclaration) {`	アノテーション型なら…
192	`            return "@interface";`	"@interface" を返す。
193	`        }`	if の終わり。
194	`        return "type";`	どれにも当てはまらない場合の保険。
195	`    }`	kindOf メソッドの終わり。
196		

⑨ 補助メソッド群：文字列の組み立て（197〜237 行）

見出し行を作るときに使う小さな部品です。どれも「StringBuilder に何かを書き足す」か「文字列を作って返す」だけです。

行	コード	説明
197	`    private static void appendAnnotations(StringBuilder sb, NodeList<AnnotationExpr> annotations) {`	アノテーションを sb に追加する補助メソッド。
198	`        for (AnnotationExpr a : annotations) {`	アノテーションを 1 つずつ取り出して…
199	`            sb.append(a).append(' ');`	書かれたとおりの文字列とスペースを追加する。
200	`        }`	for の終わり。
201	`    }`	メソッドの終わり。
202		
203	`    private static void appendModifiers(StringBuilder sb, NodeList<Modifier> modifiers) {`	修飾子を sb に追加する補助メソッド。
204	`        for (Modifier m : modifiers) {`	修飾子を 1 つずつ取り出して…
205	`            sb.append(m.getKeyword().asString()).append(' ');`	キーワード（public など）の文字列とスペースを追加する。

行	コード	説明
206	`        }`	for の終わり。
207	`    }`	メソッドの終わり。
208		
209	`    private static void appendTypeParameters(StringBuilder sb, NodeList<TypeParameter> params) {`	型パラメータを sb に追加する補助メソッド。
210	`        if (!params.isEmpty()) {`	型パラメータがある場合だけ…
211	`            sb.append('<').append(params.stream()`	「<」を書いて、型パラメータを順に処理する。
212	`                    .map(TypeParameter::toString)`	それぞれを文字列にし…
213	`                    .collect(Collectors.joining(", "))).append('>');`	「, 」でつなげ、最後に「>」で閉じる。
214	`        }`	if の終わり。
215	`    }`	メソッドの終わり。
216		
217	`    private static void appendTypeList(StringBuilder sb, String keyword, NodeList<ClassOrInterfaceType> types) {`	extends / implements / permits などの型の一覧を追加する補助メソッド。 keyword にそのキーワードを渡す。
218	`        if (!types.isEmpty()) {`	型が 1 つ以上ある場合だけ…
219	`            sb.append(keyword).append(types.stream()`	キーワードを書いて、型を順に処理する。
220	`                    .map(ClassOrInterfaceType::asString)`	それぞれを文字列にし…
221	`                    .collect(Collectors.joining(", ")));`	「, 」でつなげる。
222	`        }`	if の終わり。
223	`    }`	メソッドの終わり。
224		
225	`    private static void appendThrows(StringBuilder sb, NodeList<? extends com.github.javaparser.ast.type.ReferenceType> types) {`	throws の例外一覧を追加する補助メソッド。
226	`        if (!types.isEmpty()) {`	例外がある場合だけ…
227	`            sb.append(" throws ").append(types.stream()`	「 throws 」を書いて、例外を順に処理する。
228	`                    .map(t -> t.asString())`	それぞれを文字列にし…
229	`                    .collect(Collectors.joining(", ")));`	「, 」でつなげる。

行	コード	説明
230	`        }`	if の終わり。
231	`    }`	メソッドの終わり。
232		
233	`    private static String joinParameters(NodeList<Parameter> params) {`	引数の一覧を「型 名前, 型 名前」の文字列にして返す補助メソッド。
234	`        return params.stream()`	引数を順に処理する。
235	`                .map(p -> p.getTypeAsString() + (p.isVarArgs() ? "..." : "") + " " + p.getNameAsString())`	「型」＋（可変長引数なら「...」）＋「名前」の形にする。
236	`                .collect(Collectors.joining(", "));`	「, 」でつなげた文字列を返す。
237	`    }`	メソッドの終わり。
238	`}`	クラスの終わり。